

Deep Fake Detection

Sean Gupta, Josh Oursler, Ruixuan Hou, Haoran Zhang
University of Missouri-Kansas City

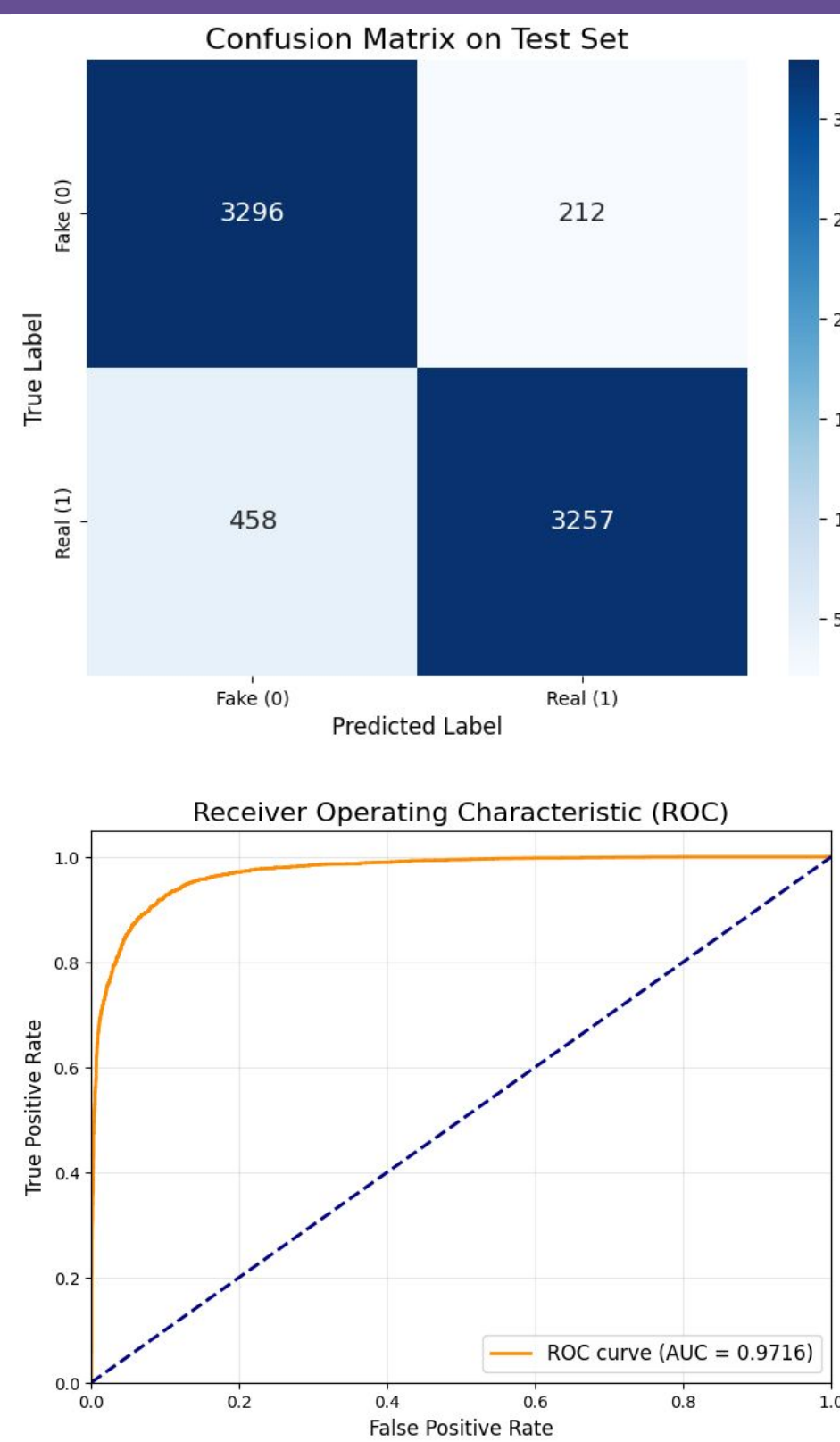
Abstract

This project develops a deep learning model to detect deep fake images. Using the OpenForensics dataset, images were pre processed through face detection, cropping, resizing, and normalization for consistency. A ResNet50 feature extractor was paired with a custom classification head to perform binary classification on the images. We implemented data augmentation to improve optimization. The model achieved an accuracy and AUC of 0.97 demonstrating strong discrimination between classes. Grad-CAM heatmap visualizations revealed that the model often focused on non-central regions for fake images showing a potential shortcut learning. While the results are strong, challenges still remain in reducing misclassification of difficult samples. This work contributes to combating the ongoing misuse of AI image generators and improving the robustness of deep fake image detection.

Introduction

Data Science is an increasingly growing field. The applications of Data Science can be used in nearly every other field. This project sought to apply the tools of Data Science to solving the problem of deep faked images of people. First, the OpenForensics Dataset was used to provide a large quantity of labeled images to facilitate a supervised learning method. Then, the photos were cropped with the cv2 library in Python so only the faces in the photos would be analyzed. The photos were then normalized to a predetermined aspect ratio before training. Finally, the model was trained on the cleaned training portion of the dataset and tested and validated on their corresponding datasets.

Accuracy



Methodology

Exploratory Data Analysis

The dataset used for this project was the OpenForensics Dataset designed by Trung-Nghia Le, Huy H. Nguyen, Junichi Yamagishi, and Isao Echizen. The dataset was designed for generating face forgery detection and segmentation models. The deep fake images were real photos that were altered through various methods. In the fake photos, some of the faces are real, but one or two are fake; therefore, the model must be able to detect individual faces and determine if that singular face is real or fake (Le).

Data Cleaning

The data was multiple sets of photos that were wrapped in a zip file. The methodology used opened the zip file in the program ensuring any user that recreates this work will not be required to purchase a program to open zip files. The folder paths for each dataset and their two categories, real or fake, were saved in variables. A function was created that opened the zip file and followed a provided path to extract a desired set of photos. The photos were copied into grayscale then processed through a face recognition function provided by the cv2 library in Python. The face recognition function returns the bounds of the face found, so the original function can create a cropped copy of the original photo around only the face found. Photos with several faces would generate several copies that were all placed in a list. This list would then be iterated through to save the photos to the clean data folder to be organized the same way as the raw data. The photos were also transformed into numpy arrays for the model to perform training and analytics on.

Data Augmentation

For training, validation, and testing, the images are transformed using a custom resize and pad function. If the image's width or height is greater than 150 pixels, the image will be resized to a maximum long edge of 150 pixels. The image's aspect ratio is maintained during the resizing process. After resizing, if there is extra space, padding will be added to the image to fill a 150 by 150 pixel space. The image is centered in the middle of the padding space. The images are then converted to tensors and normalization is applied.

In addition to the base transforms, augmentation transforms were applied only to the training data. These additional augmentations included random horizontal flip at 50% probability and random color jitter of brightness, contrast, and saturation at 20% probabilities each.

Model Structure

The model is set up using PyTorch. We used the ResNet50 as the model feature extractor. The final classification layer of the ResNet50 is removed. We added our own classification head to the model consisting of 2 linear layers separated by a ReLU function and a dropout rate of 0.2. The final output of the model is the raw logit.

Training

The dataset provides predetermined subsets for training, validation, and testing. The model was trained using the training subset provided by the dataset. Every epoch validation was completed reporting loss and accuracy on the validation subset of the data. At the end of training a single check was performed on the test subset of the data analyzing loss and accuracy.

The training loop had a check at the end of each epoch. The check was reviewing the validation loss. It saves the new best model if the performance was better than previous versions within the constraint of the min_delta hyperparameter. If a better model can't be achieved within the patience hyperparameter, the training will stop automatically.

Model Explanation Using Grad-CAM Heatmap

We used gradient-weighted class activation mapping to better understand what portions of the image the model is focusing on. We targeted the final convolutional layer in the ResNet50 feature extractor. We create a class activation map as a weighted sum of feature maps. The heatmap is then normalized and scaled to match the original image. The heatmap is then overlaid on the original image to show the regions the model is focusing on.

Results

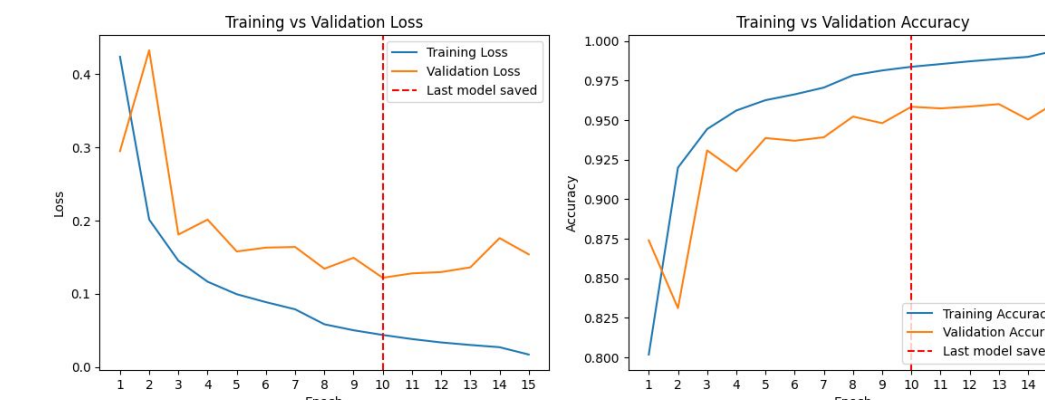
Grad-CAM Heatmap Findings

Grad-CAM is a technique that generates a heatmap showing the regions of an image that have the most influence on a convolutional network's prediction. Grad-CAM technique was used to generate heatmaps of the model inference to better understand the internal workings of the model. For real images, the model tends to focus on the central facial regions. However, for deep fake images, the model tends to focus on smaller points closer to the edges of the images that may not be part of the face. This could indicate that the model has learned some shortcut to predicting the image class. Below are several examples of the heatmaps.

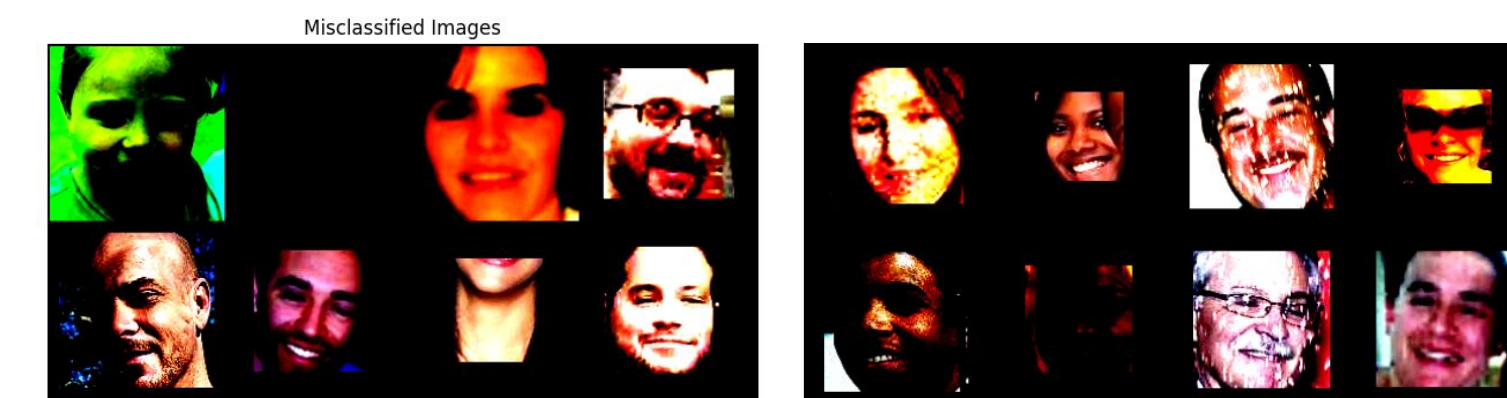


Model Accuracy & Metrics

The model achieved a loss of 0.12 and an accuracy of 95.8% on the testing dataset. The following plots show loss over epochs and accuracy over epochs respectively.



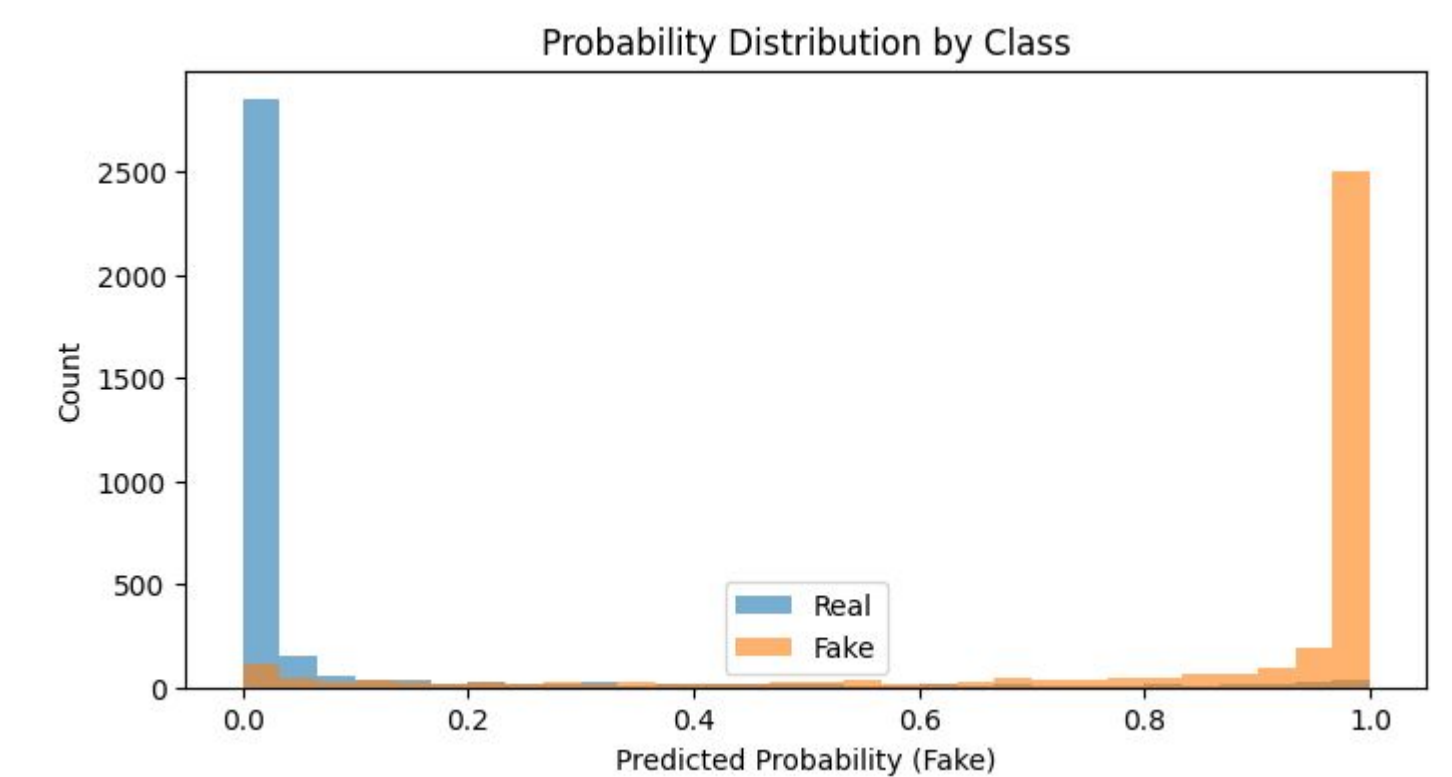
Some extra data visuals were generated to better understand the efficiency of the model, some errors in the model, and insight into the parameters of the model. In order to better understand the efficiency of the model, a probability distribution histogram was generated to visualize how well the model separated the two classes. The probability distribution histogram shows clear separation between the real and fake class with the majority of each class being on the edges. A threshold sweep curve was generated with the idea of maximizing our accuracy, precision, and recall. A threshold score of around 0.25 would maximize the three stats; however, higher threshold values could be utilized to gain more precision at approximately the same accuracy at the expense of a lower recall score. Finally, the images of people's faces that were misclassified were compiled to understand what caused our model to falsely identify them. Poor picture quality, shading around key features, and bad image crops are easily identifiable culprits for these misclassifications.



Conclusion

Our model was able to successfully detect deep fake images with high accuracy and strong discrimination between classes. The model did still struggle with a few difficult samples. Our future work includes further exploring the capabilities of our current model, exploring the Grad-CAM heatmap results to better understand why the model focuses on different points in real vs fake images, and further enhancing the model structure to achieve stronger identification on the difficult samples.

This project successfully implemented a deep fake detector to detect if an image is generated or real. We explored the image dataset for better understanding. We implemented a deep learning model for binary classification of images. We explored and explained the capabilities of the model using several techniques. This model could be used for identification of deep fake images to discover abuse of AI image generation.



Acknowledgements

Le, Trung-Nghia. "OpenForensics Dataset." Trung-Nghia Le - OpenForensics, ICCV, 2021, sites.google.com/view/ltnghia/research/openforensics.

Rössler, Andreas, et al. "FaceForensics++: Learning to Detect Manipulated Facial Images." ICCV 2019 Open Access Repository, ICCV, 2019, openaccess.thecvf.com/content_ICCV_2019/papers/Rossler_FaceForensics_Learning_to_Detect_Manipulated_Facial_Images_ICCV_2019_paper.pdf?utm.

Nataraj, Lakshmanan, et al. "Detecting GAN Generated Fake Images Using Co-occurrence Matrices." arXiv, 2019, arxiv.org/pdf/1903.06836

Xuan, Xinsheng, et al. "On the Generalization of GAN Image Forensics." arXiv, 2019, arxiv.org/pdf/1902.11153.

Wang, Sheng-Yu, et al. "CNN-Generated Images Are Surprisingly Easy to Spot... for Now." CVPR 2020 Open Access Repository, CVPR, 2020, openaccess.thecvf.com/content_CVPR_2020/papers/Wang_CNN-Generated_Images_Are_Surprisingly_Easy_to_Spot...for_Now_CVPR_2020_paper.pdf?utm.

Chai, Lucy, et al. "What Makes Fake Images Detectable? Understanding Properties That Generalize." Patch Forensics, ECCV, 2020, arxiv.org/pdf/2008.10588.

Ojha, Utkarsh, et al. "Towards Universal Fake Image Detectors That Generalize Across Generative Models." CVPR 2023 Open Access Repository, CVPR, 2023, openaccess.thecvf.com/content_CVPR2023/papers/Ojha_Towards_Universal_Fake_Image_Detectors_That_Generalize_Across_Generative_Models_CVPR_2023_paper.pdf?utm.